AMR coursework 3 Testing controller practical lab

In this lab, you will test your closed-loop controller with the Tello drone and the Vicon Motion Capture System. Vicon can track a unique pattern of reflective markers to millimetre precision, which will provide you with the feedback necessary to control the drone. Similar to previous labs, you will be working on the controller.py python script. The controller function has the following input arguments:

- state:
    - [position_x (m), position_y (m), position_z (m), roll (radians), pitch (radians), yaw (radians)]
- target_pos (target pose):
    - (x (m), y (m), z (m), yaw (radians))
- timestamp
    - timestamp in milliseconds

What you have to return:

- x velocity
    - m/s
- y velocity
    - m/s
- z velocity
    - m/s
- yaw rate
    - rad/s

You have to implement your controller inside this function. Coordinate system convention: x forward, y left, z up. The goal poses are stored in the targets.csv file which are automatically loaded.

- TELLO_VICON_NAME
    - Vicon marker object name on the system.
- TELLO_ID
    - Tello drone ID number,
- POSITION_ERROR
    - Euclidean distance from target position before going onto the next one.
- YAW_ERROR
    - Error in angle before going onto the next target.
- MAX_SPEED
    - Restricts the maximum speed of the drone.

Things to consider:

- The velocity and yaw rate control values should be within -100/100, so your controller should clamp the returned values within this range.
- The returned yaw angle from the Vicon is between -π/π.
- You may want to save some data for later analysis:

```
with open('output.csv', 'a') as f:
    new_data = np.hstack((timestamp, state))
    np.savetxt(f, [new_data], delimiter=',', fmt='%.6f')
```

- You should start with a single target pose and then move over to multi-waypoint navigation.
- You should start with higher position and yaw errors and decrease them over time after you have tested that your controller works.